

Maximum Degree-Based Quasi-Clique Search via an Iterative Framework

Hongbo Xia
Harbin Institute of Technology,
Shenzhen
Shenzhen, China
24s151167@stu.hit.edu.cn

Kaiqiang Yu
Nanyang Technological University
Singapore, Singapore
kaiqiang002@e.ntu.edu.sg

Shengxin Liu*
Harbin Institute of Technology,
Shenzhen
Shenzhen, China
sxliu@hit.edu.cn

Cheng Long
Nanyang Technological University
Singapore, Singapore
c.long@ntu.edu.sg

Xun Zhou
Harbin Institute of Technology,
Shenzhen
Pengcheng Laboratory
Shenzhen, China
zhouxun2023@hit.edu.cn

Abstract

Cohesive subgraph mining is a fundamental problem in graph theory with numerous real-world applications, such as social network analysis and protein-protein interaction modeling. Among various cohesive subgraphs, the γ -quasi-clique is widely studied for its flexibility in requiring each vertex to connect to at least a γ proportion of other vertices in the subgraph. However, solving the maximum γ -quasi-clique problem is NP-hard and further complicated by the lack of the hereditary property, which makes designing efficient pruning strategies challenging. Existing algorithms, such as DDA and FastQC, either struggle with scalability or exhibit significant performance declines for small values of γ . In this paper, we propose a novel algorithm, IterQC, which reformulates the maximum γ -quasi-clique problem as a series of k -plex problems that possess the hereditary property. IterQC introduces a non-trivial iterative framework and incorporates two key optimization techniques: (1) the *pseudo lower bound (pseudo LB)* technique, which leverages information across iterations to improve the efficiency of branch-and-bound searches, and (2) the *preprocessing* technique that reduces problem size and unnecessary iterations. Extensive experiments demonstrate that IterQC achieves up to four orders of magnitude speedup and solves significantly more graph instances compared to state-of-the-art algorithms DDA and FastQC.

CCS Concepts

• Theory of computation → Graph algorithms analysis.

*Shengxin Liu is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '25, August 3–7, 2025, Toronto, ON, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1454-2/2025/08
<https://doi.org/10.1145/3711896.3737041>

Keywords

Cohesive subgraph mining, quasi-clique, iterative framework

ACM Reference Format:

Hongbo Xia, Kaiqiang Yu, Shengxin Liu, Cheng Long, and Xun Zhou. 2025. Maximum Degree-Based Quasi-Clique Search via an Iterative Framework. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3711896.3737041>

KDD Availability Link:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.15486804>.

1 Introduction

Cohesive subgraph mining is a fundamental problem in graph theory with numerous real-world applications [13, 24, 32, 39]. Unlike cliques, which require complete connectivity, cohesive subgraphs provide more flexibility by allowing the absence of certain edges. In real-world graphs, data is often noisy and incomplete, making fully connected cliques impractical. By allowing a certain degree of edge absence, the relaxation offers a robust way to identify strongly connected substructures while accommodating missing or uncertain relationships. Various approaches have been proposed to relax the constraints of cohesive subgraph definitions, such as k -plex [54], k -core [53], and k -defective clique [63]. One widely studied definition is the γ -quasi-clique [46]. Given a fraction γ between 0 and 1, a γ -quasi-clique requires that every vertex in the subgraph is directly connected to at least a γ proportion of the other vertices in the subgraph. Cohesive subgraph mining in the context of γ -quasi-clique has recently attracted increasing interest [30, 34, 37, 40, 48, 52, 64, 68]. It has numerous applications in real-world graph structure analysis, including social network analysis [7, 24, 31] and the modeling of protein-protein interaction networks [2, 9, 55], which captures relationships between proteins. For example, in [50], researchers identify biologically significant functional groups by mining large γ -quasi-cliques that meet a minimum size threshold across various protein-protein and gene-gene interaction networks. The idea is that within a functional protein

group, most members interact frequently, suggesting a high likelihood of forming a quasi-clique [50]. Similarly, in another case study [31], γ -quasi-cliques are utilized to uncover meaningful communities within networks derived from publication data.

In this paper, we study the *maximum γ -quasi-clique* problem, which aims to identify the γ -quasi-clique with the largest number of vertices in a graph. As a natural extension of the classic maximum clique problem, this problem is unsurprisingly NP-hard [46, 48]. An even more challenging lies in the fact that the γ -quasi-clique lacks the hereditary property, i.e., subgraphs of a γ -quasi-clique are not guaranteed to also be γ -quasi-cliques. This limitation complicates the design of efficient pruning methods, unlike problems for k -plex or k -defective clique, where the hereditary property enables effective optimizations. In this paper, we focus on designing practically efficient exact algorithms to tackle this challenging problem.

Existing state-of-the-art algorithms. The state-of-the-art algorithm for the maximum γ -quasi-clique problem is DDA [48], which combines graph properties with techniques from operations research. A key feature of DDA is its iterative *enumeration (or estimation)* of the degree of the minimum-degree vertex in the maximum γ -quasi-clique. For each candidate degree, it invokes an integer programming (IP) solver multiple times to solve equivalent sub-problems and identify the final solution. Unlike other approaches relying solely on IP formulations or branch-and-bound techniques based on graph properties, DDA effectively integrates both. However, it has two limitations: (1) it naively enumerates possible minimum degree values, potentially requiring a large number of trials (e.g., DDA may enumerate $O(n)$ values, where n is the number of vertices in the graph); (2) the IP solver operates as a black box, which makes it difficult to optimize for this specific problem.

Another closely related algorithm, FastQC [64], is the state-of-the-art for enumerating all maximal γ -quasi-cliques in a graph. FastQC uses a divide-and-conquer strategy with efficient pruning techniques that leverage intrinsic graph properties to systematically and effectively enumerate all maximal γ -quasi-cliques. It is clear that the largest maximal γ -quasi-clique identified corresponds to the maximum γ -quasi-clique. However, since FastQC is not specifically designed to solve the maximum γ -quasi-clique problem, even with additional pruning methods tailored for the maximum solution, its efficiency remains suboptimal. Moreover, our experimental results in Section 5 show that the efficiency of the FastQC algorithm significantly decreases when the value of γ is relatively small.

In summary, while both DDA and FastQC have practical strengths, they fail to fully address the efficiency challenges of the maximum γ -quasi-clique problem. Their limitations in scalability and handling smaller values of γ highlight the need for more specialized solutions.

Our new methods. In this work, we propose a novel algorithm, IterQC, which reformulates the maximum γ -quasi-clique problem as a series of k -plex problems. Notably, the k -plex possesses the hereditary property, enabling the design of more effective pruning methods. To achieve this, we introduce a non-trivial *iterative framework* that solves the maximum γ -quasi-clique problem through a carefully designed iterative procedure, leveraging repeated calls to a maximum k -plex solver [15, 28, 59]. Building on this novel iterative framework, we further propose advanced optimization techniques, including the *pseudo lower bound (pseudo LB)* technique and the

preprocessing technique. The pseudo LB technique effectively coordinates information between the inner iterations and the outer iterative framework, thereby boosting the overall efficiency of the iterative procedure. Meanwhile, the preprocessing technique performs preliminary operations before the iterative framework begins, reducing both the problem size by removing redundant vertices from the graph and reducing the number of iterations by skipping unnecessary iterations. With these optimizations, our proposed algorithm IterQC equipped with these optimizations achieves up to four orders of magnitude speedup and solves more instances compared to the state-of-the-art algorithms DDA and FastQC.

Contributions. Our main contributions are as follows.

- We first propose a basic iterative framework, which correctly transforms non-hereditary problems into multiple problem instances with the hereditary property. (Section 3)
- Based on the basic iterative method, we design an improved algorithm IterQC with two key components: (1) The *pseudo lower bound (pseudo LB)* technique, which utilizes information from previous iterations to generate a pseudo lower bound. This technique optimizes the branch-and-bound search, improving performance on challenging instances. (2) The *preprocessing* technique, which computes lower and upper bounds of the optimum size to remove unpromising vertices from the graph, potentially reducing the number of iterations. (Section 4)
- We conduct extensive experiments to compare IterQC with the state-of-the-art algorithms DDA and FastQC. The results show that (1) on the 10th DIMACS and real-world graph collections, the number of instances solved by IterQC within 3 seconds exceeds the number solved by the other two baselines within 3 hours; (2) on 30 representative graphs, IterQC is up to four orders of magnitude faster than the state-of-the-art algorithms. (Section 5)

2 Preliminaries

Let $G = (V, E)$ be an undirected simple graph with $|V| = n$ vertices and $|E| = m$ edges. We denote $G[S]$ of G as the subgraph induced by the set of vertices S of V . We use $d_G(v)$ to denote the degree of v in G . Let g be an induced subgraph of G . We denote the vertex set and the edge set of g as $V(g)$ and $E(g)$, respectively. In this paper, we focus on the cohesive subgraph of γ -quasi-clique defined below.

DEFINITION 1. Given a graph $G = (V, E)$ and a real number $0 < \gamma \leq 1$, an induced subgraph g is said to be a γ -quasi-clique (γ -QC) if $\forall v \in V(g)$, $d_g(v) \geq \gamma \cdot (|V(g)| - 1)$.

We are ready to present our problem in this paper.

PROBLEM 1 (MAXIMUM γ -QUASI-CLIQUE). Given a graph $G = (V, E)$ and a real number $\gamma \in [0.5, 1]$, the Maximum γ -quasi-clique Problem (MaxQC) aims to find the largest γ -quasi-clique in G .

We first note that MaxQC has been proven to be NP-hard [46, 48]. Moreover, we let g^* be the largest γ -QC in G and $s^* = |V(g^*)|$ be the size of this maximum solution. We also note that, following previous studies [30, 37, 50, 52, 64], we consider MaxQC with $\gamma \geq 0.5$ since (1) conceptually, a γ -QC with $\gamma < 0.5$ may not be cohesive in practice (note that each vertex in such a γ -QC may connect to fewer than half of the other vertices); (2) technically, a γ -QC with $\gamma \geq 0.5$ has a small diameter of at most 2 [50].

Algorithm 1: A Basic Iterative Framework**Input:** A graph $G = (V, E)$ and a real value $0.5 \leq \gamma \leq 1$ **Output:** The size of the maximum γ -quasi-clique in G

```

1  $s_0 \leftarrow |V|$ ;  $k_1 \leftarrow \text{get-k}(s_0)$ ;  $i \leftarrow 1$ ;
2 while true do
3    $s_i \leftarrow \text{solve-plex}(k_i)$ ;
4   if  $k_i = \text{get-k}(s_i)$  then
5      $s^* \leftarrow s_i$ ; return  $s^*$ ;
6    $k_{i+1} \leftarrow \text{get-k}(s_i)$ ;  $i \leftarrow i + 1$ ;

```

3 Our Basic Iterative Framework

The design of efficient exact algorithms for MaxQC presents multiple challenges. **First**, MaxQC has been proven to be NP-hard [46, 48]. Existing studies on similar maximum cohesive subgraph problems often adopt the branch-and-bound algorithmic framework [14, 15, 59]. **Second**, the γ -quasi-clique is non-hereditary [64], i.e., an induced subgraph of a γ -QC is not necessarily a γ -QC. This non-hereditary property of the γ -QC complicates the design of effective bounding techniques within the branch-and-bound framework.

To address these challenges, our algorithm adopts an *iterative framework* that solves MaxQC by iteratively invoking a solver for another cohesive subgraph problem. The design rationale originates from the intention to *convert the non-hereditary subgraph problem into multiple instances of a hereditary subgraph problem, so as to speed up the overall procedure*.

We first introduce another useful cohesive subgraph structure.

DEFINITION 2 ([54]). *Given a graph $G = (V, E)$ and an integer $k \geq 1$, an induced subgraph g is said to be a k -plex if $\forall v \in V(g)$, $d_g(v) \geq |V(g)| - k$.*

In the literature, a related problem for k -plex is defined [3, 14, 26, 35, 36, 59, 61, 70], as shown in the following.

PROBLEM 2 (MAXIMUM k -PLEX). *Given a graph $G = (V, E)$ and a positive integer k , the Maximum k -plex Problem aims to find the largest k -plex in G .*

Based on the maximum k -plex problem, we next describe our basic iterative framework for the MaxQC problem. To simplify the description, we define the following two functions.

- $\text{get-k}(x) := \lfloor (1 - \gamma) \cdot (x - 1) \rfloor + 1$, which takes a number x of vertices as input and returns an appropriate value of k ;
- $\text{solve-plex}(y) :=$ the size of the largest y -plex in G , which takes a value y as input.

We present our basic iterative framework for solving MaxQC in Algorithm 1. Specifically, Algorithm 1 initializes s_0 as the number of vertices in the graph G , computes the corresponding value of k_1 using the function get-k , and sets the index i as 1 (Line 1). The algorithm then iteratively computes s_i by solving the maximum k_i -plex problem via the function solve-plex , while updating k_i via the function get-k in each iteration (Lines 2-6). The iteration terminates when $k_i = \text{get-k}(s_i)$ in Line 4, at which point s^* , the size of the maximum γ -quasi-clique, is returned in Line 5. Note that Algorithm 1 returns only the size of the maximum γ -quasi-clique, but can be easily modified to output the corresponding subgraph.

Correctness. We now prove the correctness of our basic iterative framework in Algorithm 1. For simplicity in the proof, we assume that the termination condition (Line 4) of Algorithm 1 is met at the $(p + 1)$ -st iteration. Then, in our iterative framework, we can obtain two sequences $\{s_0, s_1, \dots, s_p, s_{p+1}\}$ and $\{k_1, k_2, \dots, k_p, k_{p+1}\}$, where $k_{p+1} = \text{get-k}(s_{p+1})$. In our proof, we will show that the sequence of $\{s_0, s_1, \dots, s_p\}$ generated by Algorithm 1 is strictly decreasing and s_{p+1} corresponds to the largest γ -quasi-clique in the graph, i.e., $s_{p+1} = s^*$. We first consider the special case that the input graph G is already a γ -quasi-clique.

LEMMA 1. *When the input graph $G = (V, E)$ is a γ -QC, Algorithm 1 correctly returns $|V|$ as the optimum solution.*

The proof of Lemma 1, along with other omitted proofs in this section, can be found in Section A.1. In the following, we discuss the correctness of the algorithm where G itself is not a γ -quasi-clique. We first prove the properties of the sequence for s in the following.

LEMMA 2. *For the sequence $\{s_0, s_1, \dots, s_p\}$, two consecutive elements are not identical, i.e., $\forall 0 \leq i \leq p - 1$, $s_i \neq s_{i+1}$.*

LEMMA 3. *The sequence $\{s_0, s_1, \dots, s_p\}$ is strictly decreasing.*

PROOF. We prove this lemma by the mathematical induction.

① $p = 1$. As we consider the case where G itself is not a γ -quasi-clique (by Lemma 1), we have $s_p = s_1 = \text{solve-plex}(k_1) < s_0$. The base case holds true.

② $p \geq 2$. Assume that the induction holds for $p - 1$, i.e., we have $s_{p-1} < s_{p-2}$. We observe that get-k is a non-decreasing function, which implies that $\text{get-k}(s_{p-1}) \leq \text{get-k}(s_{p-2})$. Moreover, by the definition of the solve-plex function, for any $y_1 > y_2$, the inequality $\text{solve-plex}(y_1) \geq \text{solve-plex}(y_2)$ holds. Then we have

$$\begin{aligned} s_p &= \text{solve-plex}(k_p) = \text{solve-plex}(\text{get-k}(s_{p-1})) \\ &\leq \text{solve-plex}(\text{get-k}(s_{p-2})) = \text{solve-plex}(k_{p-1}) = s_{p-1}. \end{aligned}$$

Note that based on Lemma 2 the sequence $\{s_0, s_1, \dots, s_p\}$ ensures that no two adjacent elements share the same value, which implies $s_p < s_{p-1}$, completing the inductive step.

Based on ①, ②, and the principle of mathematical induction, we complete the proof of Lemma 3. $\square$

Based on Lemma 3, we can obtain the following corollary.

COROLLARY 1. *The sequence $\{s_0, s_1, \dots, s_p\}$ is finite.*

We then show the relationship between $\{s_0, s_1, \dots, s_p, s_{p+1}\}$ and the size of the largest γ -quasi-clique.

LEMMA 4. $\forall s_i \in \{s_0, s_1, \dots, s_{p+1}\}$, $s_i \geq s^*$ holds.

LEMMA 5. *Algorithm 1 correctly identifies the largest γ -quasi-clique, i.e., $s_{p+1} = s^*$.*

PROOF. We prove by contradiction. By Lemma 4, we assume, to the contrary, that $s_{p+1} > s^*$. According to the termination condition in Line 4 of Algorithm 1, we have $k_{p+1} = \text{get-k}(s_{p+1}) = 1 + \lfloor (1 - \gamma) \cdot (s_{p+1} - 1) \rfloor$. Thus, it is clear that the result obtained by $\text{solve-plex}(k_{p+1})$ is a γ -quasi-clique of size s_{p+1} , which contradicts s^* being the size of the maximum γ -quasi-clique. $\square$

Discussions. We first note that an example of Algorithm 1 and its iterative process are provided in Appendix B. As shown in Lemma 5, Algorithm 1 can correctly solve MaxQC. The time complexity analysis is simple, since there are at most n iterations (Lines 2-6) and each iteration invokes a solver for the maximum k -plex problem. Note that the current best time complexities of the maximum k -plex algorithms are $O^*((k+1)^{\delta+k-s^*})$ and $O^*(\gamma_k^\delta)$ [15, 28, 59], where O^* suppresses the polynomial factors, $\gamma_k < 2$ is the largest real root of $x^{k+3} - 2x^{k+2} + x^2 - x + 1 = 0$, and δ is the degeneracy of the graph. We also remark that prior studies adopted similar approaches by reducing a more difficult problem to solving another problem iteratively. For instance, Chang and Yao [15] briefly discuss the relationship between the γ -quasi-clique and the k -plex. They focus on enumerating maximal γ -quasi-cliques and mention a method for generating an initial solution set by enumerating maximal k -plexes, followed by a screening step to remove those that do not satisfy the maximality condition. However, this method is mainly of theoretical interest and runs slowly in practice, since it requires enumerating a large number of maximal k -plexes. Zhang and Liu [69] tackle the minimum k -core problem by reducing it to multiple iterations of solving the maximum k -plex problem. Their iterative strategy incrementally adjusts the value of k by one in each round, progressively approaching the desired solution. Similarly, they also adopt an iterative strategy that considers different values of k , where the value of k differs by one between consecutive iterations. In contrast, we consider the maximum γ -quasi-clique problem. Our method adaptively adjusts the values of k based on the size of the current best k -plex, rather than exhaustively enumerating all possible values of k .

However, Algorithm 1 still suffers from efficiency issues for the following reasons. **First**, Algorithm 1 utilizes a solver for solve-plex as a black box. This solver relies on a conservative lower bound within solve-plex to reduce the graph, leading to inefficiencies when handling certain challenging instances. **Second**, the graph processed iteratively in solve-plex (Line 3) often contains many unpromising vertices, which negatively affects overall performance, even after graph reduction within solve-plex. Further, in the initial iteration, the first value of k is set as $k_1 = \text{get-k}(|V|)$ with $|V|$ is a trivial upper bound of s^* in Line 1. This initialization could potentially result in numerous unnecessary iterations of Lines 2-6.

4 Our Improved Algorithm: IterQC

To address the limitations of our basic iterative framework (Algorithm 1), we propose an advanced iterative method IterQC (shown in Algorithm 2), which incorporates two key stages: the preprocessing stage and the iterative search stage. **First**, we consider the iterative search stage (Line 4). Within the function Plex-Search – an improved version of solve-plex from Algorithm 1 – we introduce a *pseudo lower bound* (*pseudo LB*). This technique improves practical performance when handling challenging instances while preserving the correctness of the iterative algorithm for MaxQC. The pseudo LB technique is discussed in Section 4.1. **Second**, we provide a *preprocessing* technique (Lines 1-3) to improve efficiency. Specifically, we compute both the lower and upper bounds, lb and ub , of the size s^* of the largest γ -quasi-clique g^* via Get-Bounds in Line 1. Using lb and ub , in Lines 2-3, we can (1) reduce the graph

Algorithm 2: Our Improved Algorithm: IterQC

Input: A graph $G = (V, E)$ and a real value $0.5 \leq \gamma \leq 1$

Output: The size of the maximum γ -quasi-clique in G

// Stage 1: Preprocessing

1 $lb, ub \leftarrow \text{Get-Bounds}(G, \gamma)$;

2 **while** $\min_{v \in V} d_G(v) \leq \lfloor (lb - 1) \cdot \gamma \rfloor$ **do**

3 $u \leftarrow \arg \min_{v \in V} d_G(v)$; Remove u from G ;

// Stage 2: Iterative Search

4 Improved-Iter-Search(G, γ, ub);

by removing unpromising vertices/edges from G that cannot appear in g^* , and (2) initialize a smaller value of k for Plex-Search, potentially reducing the number of iterations. The preprocessing technique is introduced in Section 4.2. Finally, we analyze the time complexity of our IterQC, which theoretically improves upon the state-of-the-art algorithms, in Section 4.3

4.1 A Novel Pseudo Lower Bound Technique

The basic iterative framework (Algorithm 1) solves MaxQC by iteratively invoking the maximum k -plex solver solve-plex as a black box with varying values of k . To improve practical efficiency, we have the following key observation.

Key observation. Existing studies [14, 15, 28, 35, 59, 61] on maximum k -plex solvers mainly consist of two steps: (1) heuristically computing a lower bound of the maximum k -plex, and (2) exhaustively conducting a branch-and-bound search to find the final result. It is well known that Step (1) has a polynomial complexity while Step (2) requires an exponential complexity. Thus, both theoretically and practically, in most cases, the time cost of the branch-and-bound search dominates that of obtaining the lower bound.

With the above property, our key idea is to *balance* the time cost of both steps. In particular, we introduce a *pseudo lower bound*, denoted by *pseudo-lb*, which is defined as the average of the heuristic lower bound from Step (1) and a known upper bound for the maximum k -plex solution. In other words, *pseudo-lb* is always at least the heuristic lower bound and at most the known upper bound. By incorporating *pseudo-lb* into the branch-and-bound search in Step (2), we may improve pruning effectiveness, which reduces the overall search time as verified by our experiments. We note that in maximum k -plex algorithms, Step (1) corresponds to Plex-Heu for computing a heuristic solution, while Step (2) corresponds to Plex-BRB for performing the branch-and-bound search.

The technique of pseudo LB. We call this technique as *pseudo lower bound* (*pseudo LB*), which is incorporated into a branch-and-bound search algorithm called Plex-Search in Algorithm 3. In Plex-Search, *ub-plex* represents an upper bound of the size of the maximum k -plex found in Plex-Search, meaning no k -plex larger than *ub-plex* exists. The output of Plex-Search include a pseudo lower bound *pseudo-lb* and a pseudo size $|S|$ which corresponds to (1) the size of the maximum k -plex of size at least *pseudo-lb* if such a k -plex exists, or (2) 0, if no k -plex of size at least *pseudo-lb* exists. This is because, when the input *pseudo-lb* exceeds the size of the maximum k -plex in the graph, Plex-BRB will utilize this *pseudo-lb* for pruning, resulting in an empty graph as the vertex set S .

Algorithm 3: Plex-Search($G, k, ub\text{-}plex$)**Output:** a pseudo lower bound $pseudo\text{-}lb$ and a pseudo size

```

1  $lb\text{-}plex \leftarrow \text{Plex-Heu}(G)$ ; //  $G$  is locally reduced.
2 if  $lb\text{-}plex = ub\text{-}plex$  then return  $lb\text{-}plex, lb\text{-}plex$ ;
3  $pseudo\text{-}lb \leftarrow \lfloor (lb\text{-}plex + ub\text{-}plex)/2 \rfloor$ ;
4  $S \leftarrow \text{Plex-BRB}(G, pseudo\text{-}lb)$ ; // The maximum  $k$ -plex
   of size at least  $pseudo\text{-}lb$  if exists; empty,
   otherwise.
5 return  $pseudo\text{-}lb, |S|$ ;

```

Algorithm 4: Improved-Iter-Search(G, γ, ub)**Output:** The size of the maximum γ -quasi-clique in G

```

1  $s_0 \leftarrow ub$ ;  $k_1 \leftarrow \text{get-}k(s_0)$ ;  $i \leftarrow 1$ ;
2 while true do
3    $pseudo\text{-}lb, pseudo\text{-}size \leftarrow \text{Plex-Search}(G, k_i, s_{i-1})$ ;
    $s_i \leftarrow \max\{pseudo\text{-}lb, pseudo\text{-}size\}$ ;
4   if  $k_i = \text{get-}k(s_i)$  and  $pseudo\text{-}size \geq pseudo\text{-}lb$  then
5      $s^* \leftarrow s_i$ ; return  $s^*$ ;
6    $k_{i+1} \leftarrow \text{get-}k(s_i)$ ;  $i \leftarrow i + 1$ ;

```

Our Plex-Search method. We describe Plex-Search in Algorithm 3. Specifically, in Line 1, we invoke Plex-Heu to compute a lower bound $lb\text{-}plex$. If $lb\text{-}plex = ub\text{-}plex$, we can directly return $lb\text{-}plex$ as both the pseudo lower bound and the pseudo size in Line 2, since Plex-Heu already finds the maximum k -plex in this case. Then, Line 3 computes our pseudo lower bound $pseudo\text{-}lb$ as the average of $lb\text{-}plex$ and $ub\text{-}plex$. We conduct Plex-BRB using $pseudo\text{-}lb$ for pruning and obtain the corresponding k -plex S (either the vertex set of maximum k -plex or an empty set). Finally, we return $pseudo\text{-}lb$ and $|S|$ in Line 5.

Our improved iterative search algorithm. With our newly proposed Plex-Search with the pseudo LB technique, we next present our improved iterative search method Improved-Iter-Search in Algorithm 4. This method is similar to our basic iterative framework in Algorithm 1. Specifically, Algorithm 4 initializes s_0 to ub , computes the corresponding value of k_1 using the function $\text{get-}k$, and sets the index i to 1 (Line 1). The algorithm then iteratively computes s_i by invoking Plex-Search (instead of solve-plex in Algorithm 1), while updating k_i using the function $\text{get-}k$ at each iteration (Lines 2-6). Note that both $pseudo\text{-}lb$ and $pseudo\text{-}size$ are obtained from our Plex-Search, and s_i is set to the greater of these two values in Line 3. The iteration terminates when $k_i = \text{get-}k(s_i)$ and $pseudo\text{-}size \geq pseudo\text{-}lb$ in Line 4. This termination condition ensures that (1) the output from Plex-Search is the maximum k -plex even with the use of a pseudo lower bound; and (2) the current iteration produces the same value of k for the next iteration from Plex-Search. At this point, we return s^* as the size of the maximum γ -quasi-clique in Line 5.

Remark. We note that our improved iterative search method in Algorithm 4 can use adaptations of any existing maximum k -plex algorithm [14, 15, 28, 35, 59, 61] for Plex-Search, provided the algorithm can be decomposed into two steps: (1) heuristically obtaining a lower bound for the maximum k -plex, Plex-Heu, and (2)

exhaustively conducting the exact branch-and-bound search for the final solution, Plex-BRB. In this work, we utilize one of the state-of-the-art maximum k -plex algorithms KPEX [28].

Correctness. We prove the correctness of our improved iterative search algorithm Improved-Iterative-Search in Algorithm 4, which includes the pseudo LB technique in Algorithm 3. The complete proof and the illustration example of the pseudo LB technique are provided in Appendix A.2 and Appendix B, respectively.

4.2 Our Preprocessing Technique

To show our preprocessing technique, we first describe the computation of lower and upper bounds Get-Bounds (Line 1 of Algorithm 2).

Computation of lower and upper bounds. Following the idea commonly adopted for computing lower and upper bounds in prior dense subgraph search studies [12, 14, 51, 64], we derive our bounds based on the k -core structure. In the following, we introduce the relevant concepts and present the bounds for our problem.

DEFINITION 3. (k -core [53]) Given a graph G and an integer k , the k -core of G is the maximal subgraph g of G such that every vertex $u \in V(g)$ has degree $d_g(u) \geq k$ in g .

Based on the k -core, a related concept is the *core number* of a vertex u , denoted as $core(u)$, which is the largest k such that u is part of a k -core in the graph. In other words, u cannot belong to any subgraph where all vertices have a degree of at least $core(u) + 1$. Let max_core be the maximum core number in the graph, i.e., $max_core = \max_{u \in V} core(u)$. Given a graph $G = (V, E)$, the core numbers for all vertices can be computed in $O(|V| + |E|)$ time using the famous peeling algorithm [6, 47], which iteratively removes vertices with the smallest degree from the current graph. We remark that during the peeling algorithm used to compute the core numbers, we can obtain the lower bound lb of s^* as a by-product. In particular, we check whether the current graph qualifies as a γ -quasi clique and record the size of the largest qualifying γ -quasi clique. Moreover, as the current graph keeps shrinking during the peeling algorithm, the lower bound lb clearly corresponds to the size of the first current graph that meets the γ -quasi clique requirements.

LEMMA 6. Given a graph G and a real number $0 < \gamma \leq 1$, we have $s^* \leq 1 + \lceil max_core/\gamma \rceil$.

The proof of Lemma 6 directly follows from the fact that the minimum degree of a γ -QC is at most max_core and the definition of γ -QC. Our method Get-Bounds for computing both lower and upper bounds is summarized in Algorithm 5. In particular, as mentioned, Algorithm 5 follows the peeling algorithm, which iteratively removes the vertex with the smallest degree from the current graph while dynamically updating ub and lb .

The preprocessing stage. After obtaining both lower and upper bounds via Get-Bounds, we show how to utilize these bounds in our preprocessing. Recall that, with lb and ub , we can (1) reduce the graph by removing unpromising vertices from G that cannot appear in g^* using lb , and (2) set a smaller initial value of k in the function Plex-Search using ub . The details are as follows. For (1), we can iteratively remove vertices from G with degrees at most $\lfloor (lb-1) \cdot \gamma \rfloor$ and their incident edges, since it is clear that no solution of size greater than lb can still include these vertices. For (2), we can easily derive from Lemma 7 (Section 4 where the proofs are

Algorithm 5: Get-Bounds(G, γ)**Output:** The lower and upper bounds lb and ub of s^*

```

1  $lb \leftarrow 0; ub \leftarrow 0; max\_core \leftarrow 0;$ 
2 while  $V(G) \neq \emptyset$  do
3    $u \leftarrow \arg \min_{v \in V(G)} d_G(v);$ 
4    $max\_core \leftarrow \max(max\_core, d_G(u));$ 
5    $ub \leftarrow \max(ub, \min(1 + \lceil max\_core / \gamma \rceil, |V(G)|));$ 
6   if  $G$  is a  $\gamma$ -quasi-clique and  $lb = 0$  then
7      $lb \leftarrow |V(G)|;$ 
8   Remove  $u$  from  $G;$ 
9 return  $lb, ub;$ 

```

in Appendix A.2) that, instead of initializing s_0 as $|V|$, s_0 can be set to an integer larger than s^* while still ensuring our iterative framework produces the correct solution. This allows us to use the obtained upper bound ub to update s_0 . Additionally, this ub is used to generate the initial value of k for Plex-Search through get- k , while maintaining the correctness of the iterative framework.

4.3 Time Complexity Analysis

As mentioned, in our implementation of Improved-Iter-Search (or specifically, Plex-Solve in Algorithm 3), we use two components, i.e., the heuristic and the branch-and-bound search, of the maximum k -plex solver kPEX [28]. The time complexity of our exact algorithm IterQC is dominated by Plex-BRB in Algorithm 3, which is invoked at most n times. The time complexities of kPEX are $O^*((k+1)^{\delta+k-s^*})$ and $O^*(\gamma_k^\delta)$, where O^* suppresses the polynomials, $\gamma_k < 2$ is the largest real root of $x^{k+3} - 2x^{k+2} + x^2 - x + 1 = 0$, and δ is the degeneracy of the graph. These complexities are derived using the branching methods in [15, 59], which represent the current best time complexities for the maximum k -plex problem. In addition, as the sequence $\{s_1, s_2, \dots, s_p\}$ generated by our improved iterative framework is monotonically decreasing and get- k is a non-decreasing function, each value of k_i is thus upper bounded by k_1 . Thus, the time complexity of each iteration depends on solving the maximum k -plex problem with k_1 . Further, with the preprocessing technique, $k_1 = \delta^{\frac{1-\gamma}{\gamma}}$ and we let $k = k_1$. Therefore, the time complexity of IterQC is given by $O^*\left(\min\left\{(\delta+1)^{2\delta-s^*}, \gamma_k^\delta\right\}\right)$. This improves upon (1) the time complexity of FastQC [64], which is $O^*(\sigma_k^{\delta \cdot \Delta})$, where Δ denotes the maximum degree of vertices in the graph and $\gamma_k < \sigma_k$ for each k , and improves upon (2) the time complexity of DDA [48], which is $O^*(2^n)$.

5 Experiments

We conduct extensive experiments to evaluate the practical performance of our exact algorithm IterQC against other exact methods.

- DDA¹: the state-of-the-art algorithm [48].
- FastQC²: a baseline adapted from the state-of-the-art algorithm for enumerating all maximal γ -QCs of size at least a given lower

¹We re-implemented the algorithm based on the description in [48] using the ILP solver IBM ILOG CPLEX Optimizer (version 22.1.0.0). Our implementation has better performances compared with the results shown in their paper.

²<https://github.com/KaiqiangYu/SIGMOD24-MQC>

Table 1: Statistics of 30 representative graphs.

ID	Graph	n	m	density	δ
G1	tech-WHOIS	7476	56943	$2.04 \cdot 10^{-3}$	88
G2	scc_retweet	17151	24015	$1.63 \cdot 10^{-4}$	19
G3	socfb-Berkeley	22900	852419	$3.25 \cdot 10^{-3}$	64
G4	ia-email-EU	32430	54397	$1.03 \cdot 10^{-4}$	22
G5	socfb-Penn94	41536	1362220	$1.58 \cdot 10^{-3}$	62
G6	sc-nasasrb	54870	1311227	$8.71 \cdot 10^{-4}$	35
G7	socfb-OR	63392	816886	$4.07 \cdot 10^{-4}$	52
G8	soc-Epinions1	75879	405740	$1.41 \cdot 10^{-4}$	67
G9	rec-amazon	91813	125704	$2.98 \cdot 10^{-5}$	4
G10	ia-wiki-Talk	92117	360767	$8.50 \cdot 10^{-5}$	58
G11	soc-LiveMocha	104103	2193083	$4.05 \cdot 10^{-4}$	92
G12	soc-gowalla	196591	950327	$4.92 \cdot 10^{-5}$	51
G13	delaunay_n18	262144	786395	$2.29 \cdot 10^{-5}$	4
G14	auto	448695	3314610	$3.29 \cdot 10^{-5}$	9
G15	soc-digg	770799	5907132	$1.99 \cdot 10^{-5}$	236
G16	ca-hollywood-2009	1069126	56306653	$9.85 \cdot 10^{-5}$	2208
G17	inf-belgium_osm	1441295	1549969	$1.49 \cdot 10^{-6}$	3
G18	soc-orkut	2997166	106349209	$2.37 \cdot 10^{-5}$	230
G19	rgg_n_2_22_s0	4194301	30359197	$3.45 \cdot 10^{-6}$	19
G20	inf-great-britain_osm	7733821	8156516	$2.73 \cdot 10^{-7}$	3
G21	inf-asia_osm	11950756	12711602	$1.78 \cdot 10^{-7}$	3
G22	hugetrace-00010	12057441	18082178	$2.49 \cdot 10^{-7}$	2
G23	inf-road_central	14081816	16933412	$1.71 \cdot 10^{-7}$	3
G24	hugetrace-00020	16002413	23998812	$1.87 \cdot 10^{-7}$	2
G25	rgg_n_2_24_s0	16777215	132557199	$9.42 \cdot 10^{-7}$	20
G26	delaunay_n24	16777216	50331600	$3.58 \cdot 10^{-7}$	4
G27	hugebubbles-00020	21198119	31790178	$1.41 \cdot 10^{-7}$	2
G28	inf-road-usa	23947347	28854312	$1.01 \cdot 10^{-7}$	3
G29	inf-europe_osm	50912018	54054659	$4.17 \cdot 10^{-8}$	3
G30	socfb-uci-uni	58790782	92208195	$5.34 \cdot 10^{-8}$	16

bound [64]. We adapt FastQC for our MaxQC problem by tracking of the largest maximal γ -QC seen so far and dynamically updating the lower bound to facilitate better pruning. Note that the largest γ -QC is the largest one among all maximal γ -QCs.

Setup. All algorithms are implemented in C++, compiled with g++-O3, and run on a machine with an Intel CPU @ 2.60GHz and 256GB main memory running Ubuntu 20.04.6. We set the time limit as 3 hours (i.e., 10,800s) and use OOT (Out Of Time limit) to represent the time exceeds the limit.

Datasets. We evaluate the algorithms on two graph collections (223 graphs in total) that are widely used in previous studies.

- The **real-world** collection³ contains 139 real-world graphs from Network Repository with up to 5.87×10^7 vertices.
- The **10th DIMACS** collection⁴ contains 84 graphs from the 10th DIMACS implementation challenge with up to 5.09×10^7 vertices.

To provide a detailed comparison, we select 30 representative graphs as in Table 1 (where density is computed as $\frac{2m}{n(n-1)}$). Specifically, these representative graphs include 10 small graphs (i.e., G1 to G10) with $n < 10^5$, 10 medium graphs (i.e., G11 to G20) with $10^5 \leq n < 10^7$, and 10 large graphs (i.e., G21 to G30) with $n \geq 10^7$.

5.1 Comparison with Baselines

Number of solved instances. We first compare our algorithm IterQC with the baselines DDA and FastQC by considering the number of instances solved within 3-hour and 3-second limit on two collections in Figure 1. In addition, we present the number of instances solved over time for both collections at γ values of 0.65,

³<http://lcs.ios.ac.cn/caisw/Resource/realworld%20graphs.tar.gz>

⁴<https://sites.cc.gatech.edu/dimacs10/downloads.shtml>

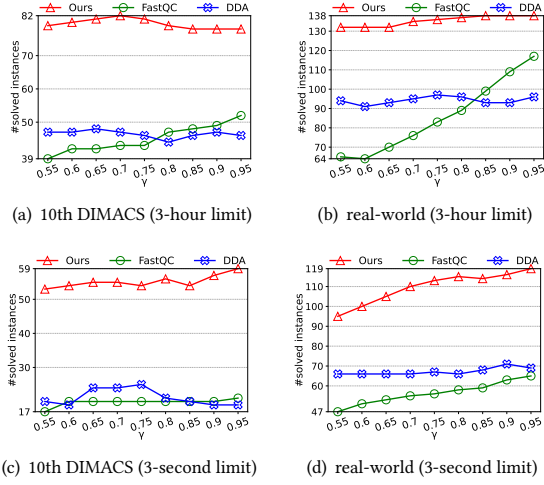
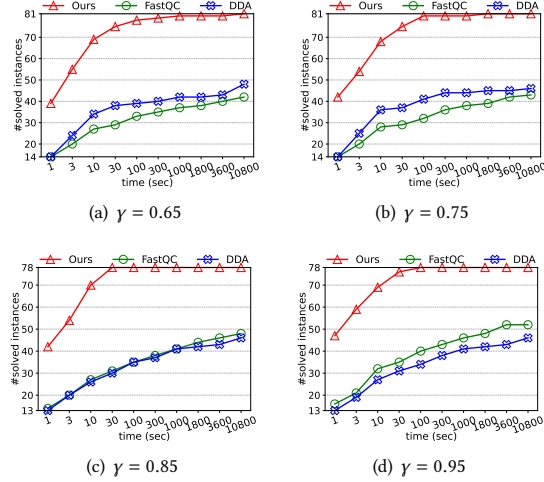
Figure 1: Number of solved instances with varying γ .

Figure 2: Number of solved instances on 10th DIMACS.

0.75, 0.85, and 0.95, in Figures 2 and 3. We have the following observations. **First**, our algorithm IterQC solves a greater number of instances across different values of γ , compared to the baselines FastQC and DDA. For example, in the real-world dataset with $\gamma = 0.7$ (in Figure 1(b)), IterQC solves 135 out of 139 instances, while DDA and FastQC only solve 95 and 76 instances, respectively. **Second**, in general, as the value of γ decreases, IterQC tends to use relatively larger values of k during each iteration, leading to higher computational costs and longer overall running times. As shown in Figures 1(b), 1(c) and 1(d), the number of instances solved by IterQC generally increases with γ . However, this trend is less apparent in Figure 1(a). Specifically, as γ decreases from 0.8 to 0.7, IterQC solves more instances. This phenomenon may be due to the following factors. IterQC computes a heuristic upper bound ub in the preprocessing stage. The gap between this ub and the optimum solution s^* is unpredictable for different values of γ . A smaller gap may result in a potentially shorter overall running time. Additionally, in our graph reduction process in the preprocessing stage,

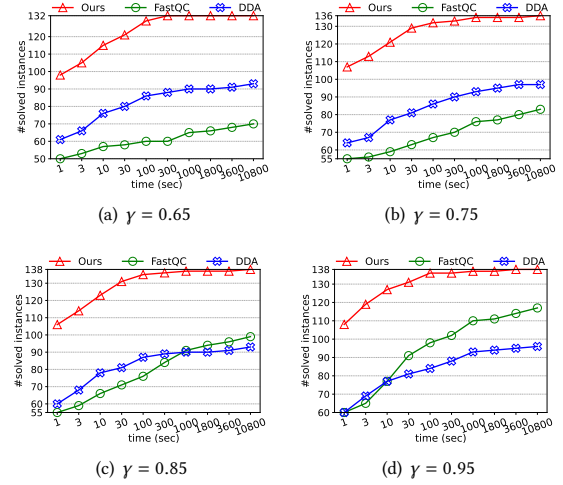
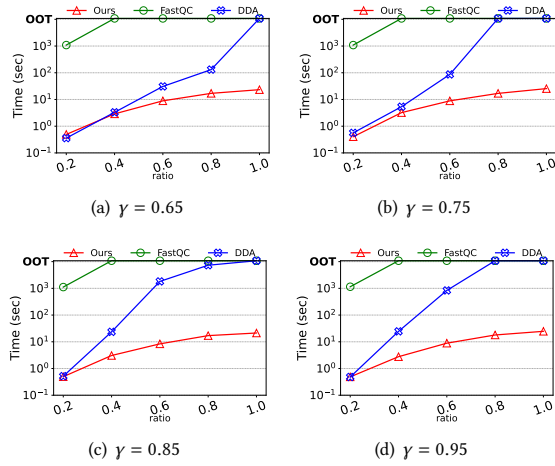


Figure 3: Number of solved instances on real-world.

a smaller γ leads to a non-decreasing lower bound lb , which enhances the effectiveness of graph reduction and reduces subsequent search costs. Moreover, the increased computational complexity associated with smaller values of γ primarily arises from the branch-and-bound search process. Intuitively, as γ decreases, the relaxation of the clique condition becomes more significant, making it harder to prune branches that could previously be terminated early. The pseudo LB technique accelerates the branch-and-bound approach and helps reduce the increased difficulty introduced by smaller values of γ . **Third**, we observe in Figures 2 and 3 that the number of instances that IterQC can solve within 3 seconds exceeds the number solved by the other two baselines within three hours. For example, on the 10th DIMACS graphs with $\gamma = 0.75$, IterQC solves 54 instances within 3 seconds, while DDA and FastQC solve 46 and 43 instances within 3 hours, respectively. Moreover, as shown in Figure 3(c), IterQC can complete 105 instances in 1 second, while DDA and FastQC finish 93 and 99 instances within 3 hours, respectively. **Performance on representative instances.** The runtime performance comparison between IterQC and the two baseline algorithms with $\gamma = 0.75$ across 30 representative instances is shown in Table 2. As illustrated in the table, IterQC consistently demonstrates superior efficiency, outperforming both baselines FastQC and DDA across nearly all instances. In particular, IterQC can solve all the graph instances, while both baseline algorithms FastQC and DDA exhibit a high occurrence of timeouts, failing to yield solutions within the 3-hour limit. Specifically, FastQC and DDA fail to solve 23 and 16 instances, respectively. Furthermore, IterQC successfully solves 14 out of the 30 representative instances where both baseline algorithms exceed the 3-hour time limit. For example, on G3, IterQC uses only 0.37 second, while both baselines cannot complete in 3 hours, achieving at least a 29,000 $\times$ speed-up. These results further suggest the efficiency superiority of IterQC over both baseline algorithms. The superior performance of IterQC, particularly compared to FastQC, may be due to the hereditary property of the k -plex, which allows for more efficient pruning during the branch-and-bound search. However, in rare cases, the computational overhead of IterQC exceeds that of DDA, such as in

Table 2: Runtime performance (in seconds) of IterQC, FastQC, and DDA on 30 instances with $\gamma = 0.75$.

ID	IterQC	FastQC	DDA	ID	IterQC	FastQC	DDA
G1	0.03	OOT	0.16	G16	5.70	OOT	OOT
G2	0.004	652.72	0.10	G17	0.24	1073.59	0.34
G3	0.37	OOT	OOT	G18	341.59	OOT	OOT
G4	0.02	4.93	69.71	G19	4.06	OOT	2.73
G5	0.91	OOT	OOT	G20	1.38	OOT	1.49
G6	4.46	80.61	OOT	G21	1.87	OOT	2.14
G7	0.24	OOT	3059.20	G22	8.70	OOT	OOT
G8	0.32	OOT	OOT	G23	3.98	OOT	3.45
G9	0.02	3.88	0.20	G24	13.07	OOT	OOT
G10	2.46	OOT	OOT	G25	18.00	OOT	9.30
G11	17.71	OOT	OOT	G26	14.86	OOT	OOT
G12	0.41	OOT	OOT	G27	18.64	OOT	OOT
G13	0.26	34.60	297.66	G28	4.75	OOT	42.54
G14	3.28	106.46	OOT	G29	9.377	OOT	9.378
G15	772.93	OOT	OOT	G30	25.50	OOT	OOT

**Figure 4: Scalability test on G30.**

G19, G23, and G26. This is because DDA adopts an iterative approach based on the IP solver CPLEX, which differs fundamentally from the branch-and-bound-based approaches used by IterQC and FastQC. As a general-purpose solver, CPLEX is not specifically optimized for the γ -quasi-clique problem and is difficult to tailor for it. Consequently, while DDA may occasionally perform better in specific instances, IterQC generally outperforms DDA in most cases.

Scalability test. We use G30 for the scalability test, which has the most vertices among the representative graphs. In our experiment, we randomly extract 20% to 100% of the vertices and test the performance of IterQC and two baselines with γ values of 0.65, 0.75, 0.85, and 0.95 in Figure 4. The results demonstrate two findings: First, in almost all cases, IterQC consistently achieves the shortest runtime. Second, across all four different values of γ , as the ratio increases—indicating a larger graph size—the increase in the runtime of IterQC is significantly smaller compared to the other algorithms. For example, in Figure 4(c), when the ratio increases from 0.2 to 0.8, the runtime of DDA rises from 0.52 seconds to 1817.60 seconds, whereas IterQC only increases from 0.49 seconds to 8.36 seconds. As for FastQC, at a ratio of 0.2, the runtime is 1086.88 seconds, but when the ratio reaches 0.4, it exceeds the time limit (10,800 seconds). These results demonstrate the scalability of IterQC.

Table 3: Runtime performance (in seconds) of IterQC, IterQC-PP, and IterQC-PLB on 30 instances with $\gamma = 0.75$.

ID	IterQC	-PP	-PLB	ID	IterQC	-PP	-PLB
G1	0.034	0.10	0.035	G16	5.70	35.54	6.31
G2	0.0043	0.039	0.0044	G17	0.24	2.96	0.32
G3	0.37	2.21	0.41	G18	341.59	4946.34	OOT
G4	0.019	0.08	0.015	G19	4.06	16.96	4.14
G5	0.91	6.64	0.80	G20	1.38	17.11	1.73
G6	4.46	6.65	4.86	G21	1.87	29.47	2.50
G7	0.24	1.36	0.17	G22	8.70	49.07	9.29
G8	0.32	1.24	0.61	G23	3.98	37.37	5.15
G9	0.021	0.17	0.018	G24	13.07	53.49	14.76
G10	2.46	2.51	29.79	G25	18.00	81.90	18.60
G11	17.71	97.73	134.01	G26	14.86	113.84	16.36
G12	0.41	1.05	0.66	G27	18.64	79.77	21.07
G13	0.26	0.99	0.22	G28	4.75	87.08	4.94
G14	3.28	11.15	3.42	G29	9.38	191.62	9.99
G15	772.93	1385.18	OOT	G30	25.50	195.29	25.97

5.2 Ablation Studies

We conduct ablation studies to evaluate the effectiveness of the techniques of preprocessing and pseudo LB proposed in Section 4. We compare IterQC with the following variants:

- IterQC-PP: it removes the preprocessing technique in IterQC, which includes (1) the initial estimation of lower and upper bounds, and (2) graph reduction. Specifically, IterQC-PP replaces Lines 1-3 in Algorithm 2 with $ub \leftarrow |V|$.
- IterQC-PLB: it removes the pseudo lower bound *pseudo-lb* and utilizes the true heuristic lower bound by replacing Line 3 in Algorithm 3 with *pseudo-lb* $\leftarrow lb_plex$.

Table 3 presents the runtime performance of IterQC, IterQC-PP, and IterQC-PLB on 30 representative instances with $\gamma = 0.75$.

Effectiveness of the preprocessing technique. From Table 3, we observe that IterQC consistently outperforms IterQC-PP, achieving a speedup factor of at least 5 in 17 instances and at least 10 in 6 instances, with a remarkable speedup factor of 20.32 on G29. We also summarize additional information for our preprocessing technique in Table 4, which details the percentages of vertices and edges pruned during preprocessing, as well as the lower and upper bounds (lb and ub) for the optimum solution s^* . From Table 4, we observe that in 3 instances (G17, G20, and G29), the preprocessing technique prunes all vertices and edges, effectively obtaining the solution directly, while in 15 instances, it removes at least 90% of the vertices. Moreover, across the 30 instances, the preprocessing technique enables the iteration process to start from a smaller initial value (closer to the optimum solution s^*), as indicated by the upper bound ub (in contrast to the trivial upper bound $|V|$ in Table 1).

Effectiveness of the pseudo LB technique. We can see in Table 3 that applying the pseudo LB technique leads to an improved performance in 25 of these 30 instances. Moreover, compared to IterQC-PLB, IterQC successfully solves 2 additional OOT instances, i.e., G15 and G18. For G18, IterQC solves in 341.59 seconds while IterQC-PLB times out, implying a speedup factor of at least 31.62. This improvement is due to the acceleration of the branch-and-bound search by the pseudo LB technique in Algorithm 3, particularly by leveraging the graph structure: dense local regions increase branch-and-bound search costs, leading to greater speedups. Conversely, instances like G4, G5, G7, G9, and G13 show lower effectiveness when the running time is dominated by the computations

Table 4: Preprocessing information with $\gamma = 0.75$, where Red-V/Red-E represent the percentages of reduced vertices/edges.

ID	Red-V (%)	Red-E (%)	lb	ub	s*
G1	98.29	87.61	116	119	117
G2	80.85	62.72	230	233	230
G3	65.86	46.16	74	87	74
G4	98.38	84.10	17	31	21
G5	60.22	39.17	57	84	66
G6	1.56	0.84	24	48	30
G7	93.32	77.40	54	71	59
G8	96.85	67.91	57	91	58
G9	98.36	97.56	6	7	6
G10	96.15	63.47	29	79	33
G11	42.27	8.76	13	124	45
G12	98.08	86.33	37	69	45
G13	0.00	0.00	3	7	6
G14	0.00	0.00	6	13	10
G15	97.49	53.99	86	316	127
G16	99.79	95.67	2209	2945	2209
G17	100.00	100.00	5	5	5
G18	76.82	52.51	66	308	130
G19	99.87	99.84	20	27	25
G20	100.00	100.00	5	5	5
G21	25.34	23.82	3	5	5
G22	0.00	0.00	2	4	2
G23	0.00	0.00	2	5	5
G24	0.00	0.00	2	4	2
G25	99.99	99.99	25	28	27
G26	1.15	1.15	5	7	6
G27	0.00	0.00	2	4	2
G28	29.35	24.36	3	5	5
G29	100.00	100.00	5	5	5
G30	99.72	98.72	10	23	10

of the heuristic lower bound in Line 4 of Algorithm 3. Despite this, even in these instances, the impact on running time is minor, with all such instances completing in under 1 second.

6 Related Work

Maximum γ -quasi-clique search problem. The maximum γ -quasi-clique search problem is NP-hard [46, 48] and W[1]-hard parameterized by several graph parameters [4, 5]. The state-of-the-art exact algorithm for solving this problem is DDA [48] and extensively discussed in Section 1. In contrast, Bhattacharyya and Bandyopadhyay [9] provided a greedy heuristic. Further studies [18, 38] addressed related problems of finding the largest maximal quasi-cliques that include a given target vertex or vertex set in a graph. Moreover, Marinelli et al. [45] proposed an IP-based method to compute upper bounds for the maximum γ -quasi-clique.

Maximal γ -quasi-clique enumeration problem. A closely related problem is the enumeration of all maximal γ -quasi-cliques in a given graph [37, 40, 64], where a γ -QC g is *maximal* if no supergraph g' of g is also a γ -QC. Several branch-and-bound algorithms have been proposed to tackle this problem by using multiple pruning techniques to reduce the search space during enumeration. In particular, Liu and Wong [40], Guo et al. [30], and Khalil et al. [37] developed such algorithms to improve efficiency. Recently, Yu and Long [64] introduced FastQC, the current state-of-the-art algorithm combining pruning and branching co-design approach. We remark that the maximum γ -QC search problem can be solved using algorithms designed for maximal γ -QC enumeration, as the

maximum γ -QC is always a maximal one in the graph. In our experimental studies, we adapt the state-of-the-art maximal γ -QC enumeration algorithm, FastQC, as the baseline method to solve our problem. Some studies explored different problem variants. For example, Sanei-Mehri et al. [52] focused on the top- k variant, Guo et al. [31] studied the problem in directed graphs, and others considered graph databases instead of a single graph [34, 68].

Other cohesive subgraph mining problems. Another approach to cohesive subgraph mining involves relaxing the clique definition from the perspective of edges. This approach gives rise to the concept of the edge-based γ -quasi-clique [1, 19, 49], which is also referred to as pseudo-cliques [58], dense subgraphs [42], or near-cliques [10, 56]. In this cohesive subgraph model, the total number of edges in a subgraph must be at least $\gamma \cdot \binom{n}{2}$. Very recently, Rahman et al. [51] introduced a novel pruning strategy based on Turán’s theorem [33] to obtain an exact solution, building on the PCE algorithm proposed by Uno [58]. Similar to the γ -quasi-clique problem, several studies have also explored non-exact approaches to solve the edge-based γ -quasi-clique problem [1, 11, 16, 41]. For instance, Tsourakakis et al. [57] proposed an objective function that unifies the concepts of average-degree-based quasi-cliques and edge-based γ -quasi-clique. There also exist many other types of cohesive subgraphs, including k -plex [14, 22, 26, 35, 36, 59–61, 70, 71], k -defective clique [12, 21, 27], and densest subgraph [44, 62]. Moreover, the topic of cohesive subgraphs has also been widely studied in other types of graphs, including bipartite graphs [17, 23, 43, 65–67], directed graphs [29], temporal graphs [8], and uncertain graphs [20]. For an overview on cohesive subgraphs, see the excellent books and survey, e.g., [13, 24, 25, 32, 39].

7 Conclusion

In this paper, we studied the maximum γ -quasi clique problem and proposed an iterative framework incorporating two novel techniques: the pseudo lower bound and preprocessing. Extensive experiments demonstrated the superiority of our algorithm IterQC over state-of-the-art methods DDA and FastQC. In future work, we aim to extend our iterative approach to other cohesive graph models.

Acknowledgments

This research is partially supported by the National Natural Science Foundation of China (No. 62102117), by the Shenzhen Science and Technology Program (No. GXWD20231129111306002), and by the Key Laboratory of Interdisciplinary Research of Computation and Economics (Shanghai University of Finance and Economics), Ministry of Education. This research is also partially supported by the National Natural Science Foundation of China (No. 62472125), the Natural Science Foundation of Guangdong Province, China (No. 2025A1515011258), and Shenzhen Sustained Support for Colleges & Universities Program (No. GXWD20231128102922001). This research is also supported by the Ministry of Education, Singapore, under its Academic Research Fund (Tier 2 Award MOE-T2EP20221-0013 and Tier 1 Award (RG20/24)). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

- [1] James Abello, Mauricio G.C. Resende, and Sandra Sudarsky. 2002. Massive quasi-clique detection. In *Proceedings of the Latin American Symposium on Theoretical Informatics (LATIN)*. 598–612.
- [2] Gary D. Bader and Christopher W. V. Hogue. 2003. An automated method for finding molecular complexes in large protein interaction networks. *BMC Bioinformatics* 4, 1 (2003), 1–27.
- [3] Balabhaskar Balasundaram, Sergiy Butenko, and Illya V. Hicks. 2011. Clique relaxations in social network analysis: The maximum k -plex problem. *Operations Research* 59, 1 (2011), 133–142.
- [4] Ambroise Baril, Antoine Castillon, and Nacim Oijid. 2024. On the parameterized complexity of non-hereditary relaxations of clique. *Theoretical Computer Science* 1003 (2024), 114625.
- [5] Ambroise Baril, Riccardo Dondi, and Mohammad Mehdi Hosseinzadeh. 2021. Hardness and tractability of the γ -complete subgraph problem. *Inform. Process. Lett.* 169 (2021), 106105.
- [6] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ algorithm for cores decomposition of networks. *CoRR* cs.DS/0310049 (2003).
- [7] Punam Bedi and Chhavi Sharma. 2016. Community detection in social networks. *Data Mining and Knowledge Discovery* 6, 3 (2016), 115–135.
- [8] Matthias Bentert, Anne-Sophie Himmel, Hendrik Molter, Marco Morik, Rolf Niedermeier, and René Saitenmacher. 2019. Listing all maximal k -plexes in temporal graphs. *ACM J. Exp. Algorithmics* 24, 1 (2019), 1.13:1–1.13:27.
- [9] Malay Bhattacharyya and Sanghamitra Bandyopadhyay. 2009. Mining the largest quasi-clique in human protein interactome. In *Proceedings of the International Conference on Adaptive and Intelligent Systems*. 194–199.
- [10] Zvika Brakerski and Boaz Patt-Shamir. 2011. Distributed discovery of large near-cliques. *Distributed Computing* 24 (2011), 79–89.
- [11] Mauro Brunato, Holger H. Hoos, and Roberto Battiti. 2008. On effectively finding maximal quasi-cliques in graphs. In *Proceedings of the International Conference on Learning and Intelligent Optimization*. 41–55.
- [12] Lijun Chang. 2023. Efficient maximum k -defective clique computation with improved time complexity. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 3 (2023), 1–26.
- [13] Lijun Chang and Lu Qin. 2018. *Cohesive Subgraph Computation over Large Sparse Graphs*. Springer.
- [14] Lijun Chang, Mouyi Xu, and Darren Strash. 2022. Efficient maximum k -plex computation over large sparse graphs. *Proceedings of the VLDB Endowment* 16, 2 (2022), 127–139.
- [15] Lijun Chang and Kai Yao. 2024. Maximum k -plex computation: Theory and practice. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 1 (2024), 1–26.
- [16] Jiejiang Chen, Shaowei Cai, Shiwei Pan, Yiyuan Wang, Qingwei Lin, Mengyu Zhao, and Minghao Yin. 2021. NuQClq: An effective local search algorithm for maximum quasi-clique problem. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 12258–12266.
- [17] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2021. Efficient exact algorithms for maximum balanced biclique search in bipartite graphs. In *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*. 248–260.
- [18] Yuan Heng Chou, En Tzu Wang, and Arbee L. P. Chen. 2015. Finding maximal quasi-cliques containing a target vertex in a graph. In *Proceedings of the International Conference on DATA*. 5–15.
- [19] Patricia Conde-Céspedes, Blaise Ngonmang, and Emmanuel Viennet. 2018. An efficient method for mining the maximal α -quasi-clique-community of a given node in complex networks. *Social Network Analysis and Mining* 8, 1 (2018), 1–18.
- [20] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, Hongzhi Chen, and Guoren Wang. 2022. Fast maximal clique enumeration on uncertain graphs: A pivot-based approach. In *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*. 2034–2047.
- [21] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023. Maximal defective clique enumeration. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 1 (2023), 1–26.
- [22] Qiangqiang Dai, Rong-Hua Li, Hongchao Qin, Meihao Liao, and Guoren Wang. 2022. Scaling up maximal k -plex enumeration. In *Proceedings of the ACM International Conference on Information & Knowledge Management (CIKM)*. 345–354.
- [23] Qiangqiang Dai, Rong-Hua Li, Xiaowei Ye, Meihao Liao, Weipeng Zhang, and Guoren Wang. 2023. Hereditary cohesive subgraphs enumeration on bipartite graphs: The power of pivot-based approaches. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 2 (2023), 1–26.
- [24] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29 (2020), 353–392.
- [25] Yixiang Fang, Kai Wang, Xuemin Lin, and Wenjie Zhang. 2021. Cohesive subgraph search over big heterogeneous information networks: Applications, challenges, and solutions. In *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*. 2829–2838.
- [26] Jian Gao, Jiejiang Chen, Minghao Yin, Rong Chen, and Yiyuan Wang. 2018. An exact algorithm for maximum k -plexes in massive graphs. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 1449–1455.
- [27] Jian Gao, Zhenghang Xu, Ruizhi Li, and Minghao Yin. 2022. An exact algorithm with new upper bounds for the maximum k -defective clique problem in massive sparse graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 10174–10183.
- [28] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, and Cheng Long. 2025. Maximum k -plex search: An alternated reduction-and-bound method. *Proceedings of the VLDB Endowment* 18, 2 (2025), 363–376.
- [29] Shuohao Gao, Kaiqiang Yu, Shengxin Liu, Cheng Long, and Zelong Qiu. 2024. On searching maximum directed (k, ℓ) -plex. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 2570–2583.
- [30] Guimu Guo, Da Yan, M. Tamer Özsu, Zhe Jiang, and Jalal Khalil. 2020. Scalable mining of maximal quasi-cliques: An algorithm-system codesign approach. *Proceedings of the VLDB Endowment* 14, 4 (2020), 573–585.
- [31] Guimu Guo, Da Yan, Lyuheng Yuan, Jalal Khalil, Cheng Long, Zhe Jiang, and Yang Zhou. 2022. Maximal directed quasi-clique mining. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1900–1913.
- [32] Xin Huang, Laks V. S. Lakshmanan, and Jianliang Xu. 2019. *Community Search over Big Graphs*. Morgan & Claypool Publishers.
- [33] Shweta Jain and C. Seshadhri. 2017. A fast and provable method for estimating clique counts using Turán’s theorem. In *Proceedings of the International Conference on World Wide Web (WWW)*. 441–449.
- [34] Daxin Jiang and Jian Pei. 2009. Mining frequent cross-graph quasi-cliques. *ACM Trans. Knowl. Discov. Data* 2, 4 (2009), 16:1–16:42.
- [35] Hua Jiang, Fusheng Xu, Zhifei Zheng, Bowen Wang, and Wei Zhou. 2023. A refined upper bound and inprocessing for the maximum k -plex problem. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 5613–5621.
- [36] Hua Jiang, Dongming Zhu, Zhichao Xie, Shaowen Yao, and Zhang-Hua Fu. 2021. A new upper bound based on vertex partitioning for the maximum k -plex problem. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 1689–1696.
- [37] Jalal Khalil, Da Yan, Guimu Guo, and Lyuheng Yuan. 2022. Parallel mining of large maximal quasi-cliques. *The VLDB Journal* 31, 4 (2022), 649–674.
- [38] Pei Lee and Laks V.S. Lakshmanan. 2016. Query-driven maximum quasi-clique search. In *Proceedings of the SIAM International Conference on Data Mining (SDM)*. 522–530.
- [39] Victor E. Lee, Ning Ruan, Ruoming Jin, and Charu Aggarwal. 2010. A Survey of Algorithms for Dense Subgraph Discovery. In *Managing and Mining Graph Data*. Springer, 303–336.
- [40] Guimei Liu and Limsoon Wong. 2008. Effective pruning techniques for mining quasi-cliques. In *Proceedings of the European Conference on Machine Learning and Knowledge Discovery in Databases (ECML/PKDD)*. 33–49.
- [41] Shuhong Liu, Jincheng Zhou, Dan Wang, Zaijun Zhang, and Mingjie Lei. 2024. An optimization algorithm for maximum quasi-clique problem based on information feedback model. *PeerJ Comput. Sci.* 10 (2024), e2173.
- [42] James Long and Chris Hartman. 2010. ODES: An overlapping dense sub-graph algorithm. *Bioinformatics* 26, 21 (2010), 2788–2789.
- [43] Wensheng Luo, Kenli Li, Xu Zhou, Yunjun Gao, and Keqin Li. 2022. Maximum biplex search over bipartite graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 898–910.
- [44] Chenhao Ma, Yixiang Fang, Reynold Cheng, Laks V. S. Lakshmanan, Wenjie Zhang, and Xuemin Lin. 2021. On directed densest subgraph discovery. *ACM Transactions on Database Systems* 46, 4 (2021), 1–45.
- [45] Fabrizio Marinelli, Alessandro Pizzuti, and Fabio Rossi. 2021. LP-based dual bounds for the maximum quasi-clique problem. *Discrete Applied Mathematics* 296 (2021), 118–140.
- [46] Hideo Matsuda, Tatsuya Ishihara, and Akihiro Hashimoto. 1999. Classifying molecular sequences using a linkage graph with their pairwise similarities. *Theoretical Computer Science* 210, 2 (1999), 305–325.
- [47] David W. Matula and Leland L. Beck. 1983. Smallest-last ordering and clustering and graph coloring algorithms. *J. ACM* 30, 3 (1983), 417–427.
- [48] Grigory Pastukhov, Alexander Veremyev, Vladimir Boginski, and Oleg A. Prokopyev. 2018. On maximum degree-based γ -quasi-clique problem: Complexity and exact approaches. *Networks* 71, 2 (2018), 136–152.
- [49] Jeffrey Pattillo, Alexander Veremyev, Sergiy Butenko, and Vladimir Boginski. 2013. On the maximum quasi-clique problem. *Discrete Applied Mathematics* 161, 1–2 (2013), 244–257.
- [50] Jian Pei, Daxin Jiang, and Aidong Zhang. 2005. On mining cross-graph quasi-cliques. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*. 228–238.
- [51] Ahsanur Rahman, Kalyan Roy, Ramiza Maliha, and Townim Faisal Chowdhury. 2024. A fast exact algorithm to enumerate maximal pseudo-cliques in large sparse graphs. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*. 2479–2490.
- [52] Seyed-Vahid Sane'i-Mehri, Apurba Das, Hooman Hashemi, and Srikanta Tirthapura. 2021. Mining largest maximal quasi-cliques. *ACM Trans. Knowl. Discov. Data* 15, 5 (2021), 81:1–81:21.

- [53] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287.
- [54] Stephen B. Seidman and Brian L. Foster. 1978. A graph-theoretic generalization of the clique concept. *Journal of Mathematical Sociology* 6, 1 (1978), 139–154.
- [55] Apichat Suratanee, Martin H. Schaefer, Matthew J. Betts, Zita Soons, Heiko Mannsperger, Nathalie Harder, Marcus Oswald, Markus Gipp, Ellen Ramminger, Guillermo Marcus, et al. 2014. Characterizing protein interactions employing a genome-wide siRNA cellular phenotyping screen. *PLoS Computational Biology* 10, 9 (2014), e1003814.
- [56] Shu Tadaka and Kengo Kinoshita. 2016. NCMine: Core-peripheral based functional module detection using near-clique mining. *Bioinformatics* 32, 22 (2016), 3454–3460.
- [57] Charalampos E. Tsourakakis, Francesco Bonchi, Aristides Gionis, Francesco Gullo, and Maria A. Tsiarli. 2013. Denser than the densest subgraph: Extracting optimal quasi-cliques with quality guarantees. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*. 104–112.
- [58] Takeaki Uno. 2010. An efficient algorithm for solving pseudo clique enumeration problem. *Algorithmica* 56, 1 (2010), 3–16.
- [59] Zhen Wang, Yong Zhou, Chuan Luo, and Minghao Xiao. 2023. A fast maximum k -plex algorithm parameterized by the degeneracy gap. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 5648–5656.
- [60] Zhengren Wang, Yi Zhou, Mingyu Xiao, and Bakhadyr Khoussainov. 2022. Listing maximal k -plexes in large real-world graphs. In *Proceedings of the ACM Web Conference (WWW)*. 1517–1527.
- [61] Minghao Xiao, Wei Lin, Yijie Dai, and Yingkai Zeng. 2017. A fast algorithm to compute maximum k -plexes in social network analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 919–925.
- [62] Yichen Xu, Chenhao Ma, Yixiang Fang, and Zhifeng Bao. 2024. Efficient and effective algorithms for densest subgraph discovery and maintenance. *The VLDB Journal* 33, 5 (2024), 1427–1452.
- [63] Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.
- [64] Kaiqiang Yu and Cheng Long. 2023. Fast maximal quasi-clique enumeration: A pruning and branching co-design approach. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, Vol. 1. 1–26.
- [65] Kaiqiang Yu and Cheng Long. 2023. Maximum k -biplex search on bipartite graphs: A symmetric-BK branching approach. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 1 (2023), 1–26.
- [66] Kaiqiang Yu, Cheng Long, P. Deepak, and Tanmoy Chakraborty. 2023. On efficient large maximal biplex discovery. *IEEE Transactions on Knowledge and Data Engineering* 35, 1 (2023), 824–829.
- [67] Kaiqiang Yu, Cheng Long, Shengxin Liu, and Da Yan. 2022. Efficient algorithms for maximal k -biplex enumeration. In *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)*. 860–873.
- [68] Zhaohao Zeng, James Wang, Liming Zhou, and George Karypis. 2007. Out-of-core coherent closed quasi-clique mining from large dense graph databases. *ACM Trans. Database Syst.* 32, 2 (2007), 13:1–13:40.
- [69] Qifan Zhang and Shengxin Liu. 2023. Efficient Exact Minimum k -Core Search in Real-World Graphs. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. 3391–3401.
- [70] Yi Zhou, Shan Hu, Minghao Xiao, and Zhang-Hua Fu. 2021. Improving maximum k -plex solver via second-order reduction and graph color bounding. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 12453–12460.
- [71] Yi Zhou, Jingwei Xu, Zhenyu Guo, Mingyu Xiao, and Yan Jin. 2020. Enumerating maximal k -plexes with worst-case time guarantee. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2442–2449.

A Omitted Proofs

A.1 Omitted Proofs in Section 3

PROOF OF LEMMA 1. When G is a γ -quasi-clique, we have $d_G(v) \geq \gamma \cdot (|V| - 1)$, $\forall v \in V$. Since $d_G(v)$ is an integer for each $v \in V$, it follows that $d_G(v) \geq \lceil \gamma \cdot (|V| - 1) \rceil \geq |V| - (\lfloor (1 - \gamma) \cdot (|V| - 1) \rfloor + 1) = |V| - k_1$. Thus, G is a k_1 -plex and $\text{solve-k}(k_1) = |V|$, which implies that $s_1 = s_0$. At this point, we have $k_1 = \text{get-k}(s_1)$, and the algorithm results in $s_1 = |V|$, which completes the proof. $\square$

PROOF OF LEMMA 2. Assume, to the contrary, that $s_i = s_{i+1}$ for $0 \leq i \leq p - 1$. Then, it follows that $k_{i+1} = \text{get-k}(s_i) = \text{get-k}(s_{i+1})$, which satisfies the termination condition in Line 4. This implies

that the algorithm terminates at the $(i + 1)$ -st iteration, thus $i = p$, leading to a contradiction to $i \leq p - 1$. $\square$

PROOF OF COROLLARY 1. By Lemma 3, the sequence is strictly decreasing. Further, it is easy to see that $s_p \geq 1$ since an induced subgraph with a single vertex is a trivial solution to the maximum k -plex problem with any $k \geq 1$. Moreover, as $\{s_0, s_1, \dots, s_p\}$ is an integer sequence, the difference between any two consecutive elements is at least 1, which implies that $p \leq s_0 = |V| = n$. $\square$

PROOF OF LEMMA 4. Recall that s^* is the size of the optimum solution. Let u be the vertex with the minimum degree in the largest γ -quasi-clique g^* . Let $\gamma^* = \frac{d_{g^*}(u)}{s^* - 1}$. Then we have $s^* = \text{solve-plex}(1 + \lfloor (1 - \gamma^*) \cdot (s^* - 1) \rfloor)$. Additionally, from the definition of a γ -quasi-clique, it also follows that $\gamma^* \geq \gamma$. We then use the mathematical induction to prove.

- ① $i = 0$. $s_i = s_0 = |V| \geq s^*$. The base case holds true.
- ② $i \geq 1$. Assume that the induction holds for $i - 1$, i.e., $s_{i-1} \geq s^*$, it follows that

$$\begin{aligned}
 s_i &= \text{solve-plex}(k_i) = \text{solve-plex}(\text{get-k}(s_{i-1})) \\
 &= \text{solve-plex}(1 + \lfloor (1 - \gamma) \cdot (s_{i-1} - 1) \rfloor) \\
 &\geq \text{solve-plex}(1 + \lfloor (1 - \gamma^*) \cdot (s_{i-1} - 1) \rfloor) \\
 &\geq \text{solve-plex}(1 + \lfloor (1 - \gamma^*) \cdot (s^* - 1) \rfloor) = s^*.
 \end{aligned}$$

Based on ①, ②, and the principle of mathematical induction, we complete the proof of Lemma 4. $\square$

A.2 Correctness Proof of Algorithm 4

We first show a property on the sequence generated by our basic iterative framework in Algorithm 1. To simplify the proofs, in the following discussion, let s_p denote the last element of the sequence $\{s_0, s_1, \dots, s_p\}$ generated by the iterative frameworks (either Algorithm 1 or Algorithm 4). It is important to highlight that, in contrast to s_p in the correctness proof of the basic iterative framework, which represents the penultimate computed value, here s_p specifically corresponds to the value at the iteration where Algorithm 3 terminates, i.e., $k_p = \text{get-k}(s_p)$.

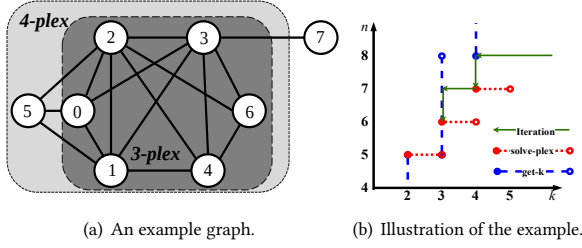
LEMMA 7. For any integer $\bar{s} \geq s^*$, the last element s_p of the sequence $\{\bar{s}, s_1, s_2, \dots, s_p\}$ generated by Algorithm 1 is equal to s^* .

PROOF. The proof is similar to the proofs of Lemmas 4 and 5, which are based on a trivial upper bound, i.e., $|V| \geq s^*$. By using $\bar{s}$ instead of $|V|$, we can prove Lemma 7. $\square$

Subsequently, in Lemma 8, we make a connection between the sequences generated by Algorithm 1 and Algorithm 4.

LEMMA 8. Consider the i -th iteration of Lines 2–6 in Algorithm 4, where $\text{Plex-Search}(G, k, \text{ub-plex})$ is called in Line 3. We let $\bar{s} \leftarrow \text{solve-plex}(k)$, i.e., $\bar{s}$ is the size of maximum k -plex. If we have $k = \text{get-k}(\text{ub-plex})$ and $\text{ub-plex} = s_{i-1} \geq s^*$, then $s_i \geq \bar{s} \geq s^*$.

PROOF. In Algorithm 3, we invoke the branch-and-bound algorithm Plex-BRB with a pseudo lower bound of pseudo-lb in Line 4, where $\text{pseudo-lb} = \lfloor (\text{lb-plex} + \text{ub-plex})/2 \rfloor$ is not the true lower bound. In other words, with pseudo-lb , the size of the returned vertex set S may not be larger than our pseudo lower bound pseudo-lb . We have two possible cases as follows.

Figure 5: An example of Algorithm 1 with $\gamma = 0.55$.

- (1) $\bar{s} > \text{pseudo-lb}$. In this case, since the pruning in Plex-BRB using *pseudo-lb* does not affect the generation of the correct solution of maximum k -plex, the returned vertex set S from Plex-BRB corresponds to the maximum k -plex. Thus, we have $|S| = \bar{s}$. Moreover, as $s_i = \max\{\text{pseudo-lb}, |S|\}$, we know that $s_i \geq \bar{s}$.
- (2) $\bar{s} \leq \text{pseudo-lb}$. In this case, in Line 4 of Algorithm 4, we have $s_i = \max\{\text{pseudo-lb}, |S|\}$, which directly implies that $s_i \geq \bar{s}$.

In either case mentioned above, we have $s_i \geq \bar{s}$. According to Lemmas 4 and 7 and $\text{ub-plex} = s_{i-1} \geq s^*$, we know that $\bar{s} \geq s^*$. The proof is thus complete. $\square$

Utilizing the connection, we verify the correctness of Algorithm 4 that uses the technique of pseudo LB in the following.

LEMMA 9. *Algorithm 4 correctly finds the largest γ -quasi-clique.*

PROOF. According to Lemma 8, we can guarantee that $s_i \geq s^*$ for each iteration i in Lines 2-6 of Algorithm 4. Thus, we can view Algorithm 4 as a procedure that continuously seeks upper bounds for the maximum γ -quasi-clique across all iterations. Let s_p denote the last element of the sequence $\{s_0, s_1, \dots, s_p\}$ generated by Algorithm 4. Next, we prove that s_p is equal to s^* .

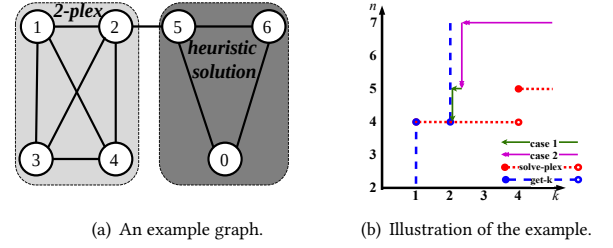
Consider the i -th iteration in Lines 2-6 of Algorithm 4, where $\text{Plex-Search}(G, k, \text{ub-plex})$ is called in Line 3. Note that $\text{ub-plex} = s_{i-1}$. We focus on this Plex-Search and have two cases.

- (1) $\text{lb-plex} = \text{ub-plex}$. Since $\text{ub-plex} = s_{i-1}$, it follows that $\text{ub-plex} = s_{i-1} \geq s^*$. We also know that there exists a k -plex of size lb-plex computed in Line 1 of Algorithm 3. Thus, we have a k -plex of size s_{i-1} , where $k = \text{get-k}(s_{i-1})$. Similar to Lemma 5, it is easy to conclude that the returned result in Line 5 of Algorithm 4 is exactly s^* . Thus, $s^* = s_{i-1} = s_i$.
- (2) $\text{lb-plex} < \text{ub-plex}$. It is easy to see that $\text{pseudo-lb} < \text{ub-plex}$. If $s_i = \text{pseudo-lb}$ in Line 3 of Algorithm 4, it guarantees that $s_i < s_{i-1}$. Otherwise, i.e., $s_i = \text{pseudo-size}$, we use $\bar{s}_i$ to denote the size of the largest k -plex in the graph G , according to the proof of Lemma 8, $s_i = \bar{s}_i$, which still satisfies $s_i = \bar{s}_i < s_{i-1}$. Thus, in any case, we have $s_i < s_{i-1}$.

Since $\{s_0, s_1, \dots, s_p\}$ is an integer sequence with each $s_i \geq s^*$, the case in 2) is always finite. In other words, the case in 1) will definitely occur. The proof is complete. $\square$

B Omitted Examples

Example of the basic iterative framework. We present an example of Algorithm 1 and illustrate its iterative process in Figures 5(a) and 5(b), respectively. In this example with $\gamma = 0.55$, the graph has 8 vertices, which means $s_0 = 8$. By applying $k_1 =$

Figure 6: An example of Algorithm 4 with $\gamma = 0.75$.

$\text{get-k}(s_0) = \lfloor (1 - 0.55) \cdot (8 - 1) \rfloor + 1 = 4$, the function *solve-plex* identifies the largest 4-plex, highlighted by the light-colored box in Figure 5(a), which results in $s_1 = 7$. Subsequently, $k_2 = \text{get-k}(s_1) = \lfloor (1 - 0.55) \cdot (7 - 1) \rfloor + 1 = 3$, and solving *solve-plex* produces the largest 3-plex, indicated by the dark-colored box in Figure 5(a), with $s_2 = 6$ vertices. At this point, the stopping condition $k_2 = \text{get-k}(s_2)$ is satisfied, and the iteration terminates. The maximum 3-plex shown in Figure 5(a) corresponds to the desired maximum 0.55-quasi-clique. In Figure 5(b), the horizontal axis represents k , while the vertical axis represents n . Moreover, (1) the sparse light blue dashed line shows the relationship between n and the corresponding k obtained from *get-k*; (2) the dense red dashed line shows the relationship between k and the resulting n obtained from *solve-plex*; (3) the green solid line represents the overall iterative process of Algorithm 1, demonstrating how n evolves from the initial graph of size 8 to the final result of the maximum γ -quasi-clique with size 6. Specifically, we start with $s_0 = 8$. After the first iteration, applying $\text{get-k}(s_0)$ transitions to the state to $(4, 8)$ in Figure 5(b) on the corresponding axes (i.e., $k_1 = 4, s_0 = 8$). Then, solving $s_1 = \text{solve-plex}(k_1)$ transitions the state to $(4, 7)$, completing one full iteration. Repeating this process leads to termination at $(3, 6)$, where the corresponding vertical coordinate 6 represents the optimum solution $s^* = 6$.

Example of the pseudo LB technique. Figure 6(a) shows a graph with 7 vertices and Figure 6(b) presents the corresponding iterative process. For clarity, line segments near $k = 2$ in Figure 6(b) are slightly shifted along the positive k -axis; this is solely for visualization and does not indicate non-integer k values. Assume that the heuristic solution obtained in each iteration corresponds to the subgraph in the dark-colored box, i.e., $G[\{0, 5, 6\}]$, in Figure 6(a), where $\text{lb-plex} = 3$ according to Line 1 of Algorithm 3. As shown in Figure 6(b), using the pseudo LB technique to solve the problem with $\gamma = 0.75$ in Figure 6(a) requires two iterations. The first iteration, represented by the double-headed purple solid line in the figure, corresponds to the second case discussed in the proof of Lemma 8. At this stage, $\text{pseudo-lb} = \lfloor (3 + 7)/2 \rfloor = 5 > \bar{s}_1 = 4$. Consequently, the branch-and-bound search in Line 4 of Algorithm 3 fails to find a solution, resulting in $S = \emptyset$. Thus, $s_1 = \max\{5, 0\} = 5$, which initiates the second iteration. In the second iteration, $\text{pseudo-lb} = \lfloor (5 + 3)/2 \rfloor = 4 \leq \bar{s}_2 = 4$, which corresponds to the first case discussed in the proof of Lemma 8. Since the pruning during the branch-and-bound search uses $\text{pseudo-lb} = 3$, it does not affect the computation of the solution S . Thus, $s_2 = \max\{3, 4\} = 4$, satisfying the condition in Line 4 of Algorithm 3. The final solution, i.e., $G[\{1, 2, 3, 4\}]$, is in Figure 6(a) within the light-colored box.